FOL: Bridging Object-Oriented and Functional Programming via the Metaobject Protocol

Frank Adrian
Ancar Technology
Olathe, KS, USA
frank.adrian314159@gmail.com

Abstract

Hickey [19] omitted CLOS from Clojure's design, viewing it as a complexity hazard incompatible with immutability. FOL (Functional Object Lisp) challenges this by using the Common Lisp Metaobject Protocol (MOP) to back every class instance with persistent data structures, preserving the full CLOS programming model while enforcing value semantics. FOL contributes (1) a persistent metaclass system with hybrid storage and lazy schema migration, and (2) a specificity-ordered predicate dispatch model.

Persistent slot reads incur 1.2–2.0× overhead; slot updates cost 30–250× in isolation. At typical workload scales, FOL is 5–47× slower than mutable CLOS; however, the fixed $O(\log n)$ structural-copy cost maintains stable throughput under sustained load, and at extreme scale the persistent heap outperforms a mutable baseline whose heap fragments, yielding a 3.8× runtime advantage in our logic-simulation benchmark. Predicate dispatch uses a level-based ordering decidable at load time without theorem-prover calls ($O(N \log N)$ vs. at least $O(N^2)$ for subsumption-based systems); closed `defn` dispatch matches hand-written `cond` cascade performance.

CCS Concepts

• **Software and its engineering → Constraints**; **Classes and objects**; **Functional languages**; **Object oriented languages**; *Extensible languages.*

Keywords

Functional programming, object-oriented programming, Lisp, persistent data structures, CLOS, MOP

1 Introduction

Common Lisp's object system (CLOS) offers powerful extensibility, but its mutable objects create three concrete problems in practice.

P1 — Speculative update cost. Speculative or concurrent workloads that may discard updates must copy object trees or maintain undo logs. Persistent alternatives (Sycamore [9], FSet [5]) operate on collections, not CLOS instances; integrating them requires abandoning generic-function dispatch. Slot updates cost 30–248× in isolation, but the fixed $O(\log n)$ copy cost maintains stable throughput under load (Section 5.5).

P2 — Schema evolution labor. Renaming slots requires a hand-written `update-instance-for-redefined-class` method; the standard hook provides no automatic forwarding. FOL's `:alias` annotation recovers renamed slots at first post-redefinition access, composing multi-hop chains in a single pass, at ≈2× first-touch cost (Section 5.2.2).

P3 — Inexpressive predicate dispatch. Standard CLOS dispatch is limited to types; value-level conditions require hand-written `cond` guards, losing extensibility. Libraries such as `filtered-functions` [27] extend this with logical subsumption but require $O(N^2)$ theorem-prover calls. FOL's level-based ordering (predicates > types > destructuring > wildcards) is decidable at load time in $O(N \log N)$; per-invocation overhead matches a hand-written `cond` cascade (≈1.6× vs. CLOS; Section 5.2.1).

FOL (Functional Object Lisp) addresses all three problems via the MOP [21]: every class instance is backed by persistent data structures, and specificity-ordered predicate dispatch integrates with standard CLOS method combinations. Slot mutation is replaced by functional `assoc`/`dissoc`; all other CLOS interfaces behave as usual. The contributions compose: Listing 4 (Section 4) shows a predicate specializer gating on a runtime value while reading persistent slot state; Listing 3 shows $O(1)$ update discard enabled by the pre-update snapshot being an immutable value.[1]

The contributions of this paper are:

(1) A **persistent metaclass system** (Section 2) with a hybrid native/trie storage strategy and lazy alias-based schema migration via multi-hop chain replay (Section 4.2).
(2) A **specificity-ordered predicate dispatch model** (Section 4): a level-based ordering relation that is decidable at load time without theorem-prover calls, with integration into standard CLOS method combinations.

2 Architecture

2.1 Object Implementation

Slot storage uses a hybrid strategy: objects with ≤ 8 slots use native CLOS slots, while wider objects overflow into a 32-way persistent trie. This threshold keeps ≈80% of classes on the native fast path, based on the class-width distribution reported by Lanza [22] for Smalltalk and Java systems; we assume this distribution generalises to CL, though it has not been independently validated for CLOS hierarchies. Table 7 shows $T = 8$ also minimizes modeled update cost for the 8-slot case, meeting both objectives. All redefinitions are handled lazily at the next object mutation (`assoc`,`dissoc`, etc.).

The `persistent-class` metaclass (Table 1) enforces: *after* `initialize-instan` *returns, no slot value is modifiable through any standard CLOS access path*; `(setf slot-value)`, `slot-makunbound`, `with-slots` mutation, and `change-class` all raise runtime errors. Standard CL code can read persistent slots via `slot-value` or generated accessors;

[1] FOL addresses the *new-operations* dimension of the Expression Problem [32]: new generic functions can be defined over existing classes without modification; adding a new data type with distinct behavior may require new method clauses.

mutation paths raise the same errors. `assoc`/`dissoc` return a new object sharing structure with the original.

A reserved `%metadata` slot stores non-structural annotations; objects differing only in metadata are = and hash identically.

2.2 Collections and Runtime

FOL implements persistent **Vectors** (32-way tries) [25], **Maps/Sets** (32-way HAMTs) [3], and **Sorted collections** (32-way B-trees); branching factor 32 ensures depth ≤ 5 for million-element collections and is optimal for update and lookup. For keyword keys, HAMT lookup is ≈1.3× faster than `gethash`,[2] as EQL comparison avoids the general hashing path.

FOL transpiles to Common Lisp (compiled by SBCL) via four phases: **reader expansion**, **macro expansion**, **code generation**, and **compilation**.

2.3 Space Complexity

FOL collections and identity types (Atoms, Refs, Agents) hold only the current value; old snapshots are GC-eligible as soon as no live reference remains.[3] Each `assoc` copies one root-to-leaf path—$O(\log_{32} N)$ new nodes, not $O(N)$. Short-lived intermediates—loop variables, traversal temporaries—are reclaimed in SBCL's nursery, keeping GC overhead at 1.5% at the scale of our largest benchmark (1.18 billion gate evaluations).

3 Features

FOL adopts Clojure's literal syntax and sequence APIs and provides: **Transducers** [20] decouple algorithm logic from collection type. **Lazy sequences** defer computation, enabling infinite sequences. **Atoms** provide compare and swap-based mutable references; **Refs** provide software transactional memory with optimistic concurrency and retry; **Agents** handle asynchronous state updates. **Transient builders** reduce allocation overhead for bulk updates by confining mutation to a thread-local window; W successive writes cost $O(W \log_{32} N)$ time but allocate only $O(\log_{32} N)$ nodes (one initial path copy). **Macros** follow Clojure-style expansion.

Generic functions dispatch by arity, then specificity (Section 4). **Closed dispatch** (`defn`) emits a `defun` with a static `cond` cascade; the clause set is fixed at compile time and closed to extension. **Open dispatch** (`defmethod`) supports CLOS method combinations and downstream clause addition. Because `defn` occupies the Lisp-2 function slot and `defmethod` defines a generic function, using both forms on the same name will cause the `defun` to shadow the generic function; programmers must choose one form per name. Table 2 summarises the decision criteria.

4 Predicate Dispatch

Figure 1 gives the formal grammar for FOL's multi-pattern dispatch syntax.

4.1 Dispatch Ordering Rules

FOL extends pattern matching with predicate specializers (`var (fn args...)`) that evaluate as `(fn var args...)` at dispatch time, gating the clause on a runtime condition. `(x (even?))` is more specific than `(x <integer>)`: it constrains value space beyond class membership, so FOL orders predicates above types. Specificity categories (Table 3) order patterns by constraint strength (Level 3 > 2 > 1 > 0), resolving same-level ties via definition order. The ordering $\prec_{\text{disp}}$ is (i) decidable at load time without theorem-prover calls, and (ii) a conservative approximation of semantic specificity: every Level-3 predicate fires before every Level-2 type, even when the predicate's extension is a superset (e.g., `(x (some?))` fires before `(x <integer>)`). Programmers should pair each Level-3 predicate with an explicit type guard—`(and (<integer>) (even?))` receives Level 3 (max rule) while making the domain explicit—rather than relying on the structural convention alone. Within Level 2, the single-parameter case uses semantic subclass ordering via the class hierarchy DAG (Spec-Type); for multi-parameter patterns the ordering is first-parameter-dominant lexicographic (Spec-Prefix/Spec-Tail), not full semantic subsumption. Within Level 1, patterns are ordered by arity containment (Spec-Length/Spec-Fixed). For example consider the following code:

```
(defn classify
  ([x <integer>]  :any-integer)   ; intended: catch all integers
  ([x (even?)]    :even))         ; intended: special case for even
                                  ; numbers
```

FOL reorders this to `even?` first:

```
(cond
  ((even? x)                    :even)
  ((typep x (find-class '<integer>)) :any-integer))
```

Even integers now never reach `:any-integer`. FOL's `even?` returns `nil` for non-integers, so the reordered guard is safe. The fix is to write clauses matching the fixed ordering:

```
(defn classify
  ([x (even?)]    :even)          ; catches even integers
  ([x <integer>]  :other-int))    ; catches remaining integers
```

Shadowed clauses are detected only at runtime; this deliberately trades theorem-prover exactness for an algorithmically bounded compilation phase.

This deliberate reliance on definition order has known limitations: methods spread across files require reasoning about ASDF load order; REPL-added methods are appended in session order with no stability guarantee across fresh loads. We are exploring explicit ordering mechanisms—analogous to Clojure's `prefer-method`—for a future version.

All method qualifiers (`:before`/`:after`/`:around`) follow $\prec_{\text{disp}}$; standard CLOS execution semantics (most-specific-first for `:before`, least-specific-first for `:after`) are preserved.

For compound predicates, the level calculus applies $\mathcal{L}(\text{and } p_1 \ldots p_n) = \max_i \mathcal{L}(p_i)$: a conjunction of type predicates is Level 2 (static), while a mixed conjunction is Level 3 (dynamic). This is semantically conservative but sound. Disjunction and negation (`or`, `not`) are not supported as dispatch combinators; predicates requiring them must be encoded as a single Level-3 function. Conjunction is supported because its level is bounded by max: the result is at least as specific as its most dynamic component, giving a sound syntactic upper bound.

Known deficiency — type-annotated conjunctions. `(and (<integer>) (prime?))` receives Level 3 ($\max(2, 3) = 3$), the same

[2] Mean of 10^6 single-key lookups on a 1,000-entry map, SBCL 2.6.0 x86-64.

[3] User-managed versioning is available: callers may hold any prior `assoc` root; structural sharing ensures retained versions cost only $O(\log_{32} N)$ extra nodes.

Table 1: MOP Protocol Adaptation

Protocol	Status	Rationale
`slot-value-using-class`	Redirected	Reads from base object or overflow trie
`(setf slot-value-using-class)`	Guarded	Blocks post-init writes
`slot-boundp-using-class`	Redirected	Checks object slot membership
`slot-makunbound-using-class`	Blocked	Immutability invariant — use `dissoc` to unbind slots
`validate-superclass`	Extended	Cross-metaclass mixing
`initialize-instance`	Extended	Populates base object and overflow trie from initargs; stamps `%schema-version` at birth
`reinitialize-instance`	Extended (`:before` on metaclass)	Snapshots current alias-map, overflow indices, and version counter into a `class-version` struct before each redefinition; advances the version counter
`compute-slots`	Specialized	Implements hybrid native/trie layout
`update-instance-for-redefined-class`	Implemented (`:after`)	Multi-hop chain replay: composes alias-maps from version chain back to birth version, recovering renamed values (Sec. 4.2)
`change-class`	Omitted	Changing a live object's class violates immutability; callers must construct a new instance of the target class instead

Method combination uses standard CLOS execution semantics (`:before`/`:after`/`:around`), with method ordering determined by $\prec_{disp}$ in place of the class precedence list.

defn ::= `(defn` *name* (*s-clause* | *m-clause*)) | `(fn` (*s-clause* | *m-clause*))
defg/m ::= `(defgeneric` *name*`...)` | `(defmethod` *name qualifier*? (*s-clause* | *m-clause*))
s-clause ::= `[`*param**`]` *body*+
m-clause ::= `{(`*symbol param* | `:keys [(`*symbol* | `(`*sym param*`))`*`])`*`}` *body*+
param ::= *symbol* | *type* | `(`*symbol type*`)` | `(`*symbol* `(`*fn arg**`))` | *destr*
destr ::= `[`*param**`]` | `{:keys [(`*symbol* | `(`*sym param*`))`*`]}`
type ::= <*type-name*>
qualifier ::= `:before` | `:after` | `:around`

Figure 1: FOL Multi-Pattern Dispatch Grammar (compressed)

Table 2: Choosing `defn` vs. `defmethod`

Requirement	`defn`	`defmethod`
Full clause set known at definition time	✓	✓
Downstream code may add clauses	×	✓
`call-next-method` needed	×	✓
`:before`/`:after`/`:around` qualifiers	×	✓
Minimum dispatch overhead (static cond)	✓	×

Table 3: Predicate Specificity Categories

Level	Category	Example
3	Predicates	`(x (even?))`
2	Types	`(x <integer>)`
1	Destructuring	`[x y]`
0	Wildcard	`x`

level as the bare predicate `(prime?)` alone, even though the conjunction is strictly more constrained. The syntactic calculus cannot determine this semantic subsumption without a theorem prover. The only current mitigation is definition order: place the conjunction clause first. We regard this as a design deficiency; a future composite level for type-annotated predicates (e.g., `(x <integer> (prime?))`) could recover automatic ordering without a theorem prover.

4.1.1 Formal Specificity Ordering. We formalize the specificity relation $\prec$ using the inference rules summarized in Table 4. Let $\mathcal{L}(p) \in \{0, 1, 2, 3\}$ be the specificity level of pattern p.

Spec-Type handles within-Level-2 ordering via the class hierarchy DAG, delegating type subsumption to the existing CPL.

The operational dispatch order $\prec_{\text{disp}}$ resolves ties using definition order. Let $idx(p)$ denote the definition index of the method clause containing p.

$$p_1 \prec_{\text{disp}} p_2 \iff p_1 \prec p_2 \vee (\neg(p_2 \prec p_1) \wedge idx(p_1) < idx(p_2))$$

Patterns of different arity are incomparable; dispatch selects the arity group first, then applies $\prec_{\text{disp}}$, which is total on same-arity patterns (Lemma 1). Definition-order tie-breaking is a pragmatic choice that aligns with Lisp's interactive workflow but introduces the modularity hazards discussed in Section 4.1.5.

Table 4: Specificity Inference Rules

Rule	Formalization
Spec-Level	$\dfrac{\mathcal{L}(p_1) > \mathcal{L}(p_2)}{p_1 \prec p_2}$
Spec-Type	$\dfrac{C_1 \sqsubset C_2}{(x\ C_1) \prec (x\ C_2)}$
Spec-Prefix	$\dfrac{p_1 \prec q_1}{[p_1, \dots] \prec [q_1, \dots]}$
Spec-Tail	$\dfrac{\mathcal{L}(p_1) = \mathcal{L}(q_1) \qquad [p_2, \dots] \prec [q_2, \dots]}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n]}$
Spec-Length	$\dfrac{\mathcal{L}(p_i) = \mathcal{L}(q_i)\ \forall i \in \{1 \dots n\}}{[p_1, \dots, p_n, p_{n+1}] \prec [q_1, \dots, q_n]}$
Spec-Fixed	$\dfrac{\mathcal{L}(p_i) = \mathcal{L}(q_i)\ \forall i \in \{1 \dots n\}}{[p_1, \dots, p_n] \prec [q_1, \dots, q_n\ \&\ r]}$
Spec-Map-Prefix	$\dfrac{p_1 \prec q_1}{\{k_1\ p_1, \dots\} \prec \{k_1\ q_1, \dots\}}$
Spec-Map-Tail	$\dfrac{p_1 = q_1 \qquad \{k_2 \dots\} \prec \{k_2 \dots\}}{\{k_1\ p_1, k_2\ p_2, \dots\} \prec \{k_1\ q_1, k_2\ q_2, \dots\}}$
Spec-Keys	$\dfrac{\text{keys}(q) \subsetneq \text{keys}(p)}{p \prec q}$

Lemma 4.1 (Totality). *$\prec_{disp}$ is a strict total order on same-arity patterns.*

Proof. $\prec$ is irreflexive: every rule requires a strict inequality (level, element position, key set, or subclass) that is false when $p_1 = p_2$. $\prec$ is transitive: the rules encode a lexicographic comparison on level vector, arity, and key set; transitivity follows by induction on pattern length, with Spec-Keys inheriting from set-containment transitivity. Incomparability under $\prec$ arises in two cases: (i) patterns differing only in variable names, and (ii) mixed sequential/map patterns at the same arity. In both cases, $\prec_{\text{disp}}$ falls back to *idx*, a monotonically increasing counter assigned at evaluation time (re-evaluation increments it), so no two live clauses share an index and $\prec_{\text{disp}}$ is total. Case (ii) is a modularity hazard: sequential and map clauses for the same generic function from different libraries are ordered solely by ASDF load sequence. □

4.1.2 Dispatch Performance. Predicate dispatch requires an $O(N)$ linear scan per call, as Level-3 resolution cannot be statically cached. Methods are sorted once at load time in $O(ND \log N)$ where D is the class hierarchy depth (each Spec-Type comparison traverses the CPL); for shallow hierarchies D is a small constant. Theorem-prover-based systems require pairwise subsumption checks at load time (at least $O(N^2)$ per generic function) but produce faster cached dispatch. To mitigate runtime cost, FOL partitions methods by arity group, ensuring calls only scan matching-arity methods.

4.1.3 Dispatch Examples. Listing 1 shows a two-clause `defn`: the compiler emits a `defun` with a specificity-ordered `cond` cascade resolved statically at compile time.

Listing 1: Transpilation: specificity-ordered multi-clause `defn`

```
;; FOL source: type specializer (Level 2) before wildcard (Level 0)
(defn classify
  ([x <integer>] :integer)
  ([x]           :other))

;; Generated Common Lisp: ordering preserved as cond cascade
;; (class references bound once via load-time-value)
(defun classify (x)
  (cond
    ((typep x (load-time-value (find-class '<integer>))) :integer)
    (t                                                   :other)))
```

Comparison to `defmulti`. Clojure's `defmulti` [16] dispatches on the return value of a user-supplied dispatch function, with ambiguity resolved manually via `prefer-method`. FOL's load-time-decidable total order $\prec_{\text{disp}}$ eliminates this burden for patterns differing in type or structural depth; the residual hazard—incomparable Level-3 predicates, discussed in Section 6—has the same scope as `prefer-method` but is narrower because structural and type differences are resolved by the level ordering.

4.1.4 Usage Patterns. Four patterns illustrate the combination of CLOS features and predicate dispatch: structural diff (Listing 2); validated update guard (Listing 3); predicate dispatch on persistent state (Listing 4); and content routing (Listing 5).

Listing 2: Automatic Structural Diff (FOL)

```
(defmethod assoc :around [(obj <diffable>) key val]
  (if (= key :_changes)
      (call-next-method)          ; :_changes key routes to primary,
                                  ; no re-entrance
      (bind [old (get obj key)
             new-obj (call-next-method)]
        (if (= old (get new-obj key))
            new-obj
            (assoc new-obj :_changes (inc (get obj :_changes 0)))))))
```

Listing 3: Validated Update Guard (FOL)

```
(defmethod assoc :around [(obj <guarded>) key val]
  (if (valid-update? obj key val)
      (call-next-method)
      obj))  ; discard invalid update; original snapshot unchanged
```

Listing 4: Predicate Dispatch on Persistent State (FOL)

```
;; Level-3 predicate (critical?) tests the reading value at dispatch
       time;
;; the body reads persistent slot :alert-history from the persistent
       sensor
;; object without copying.  Both contributions are required:
      predicate
;; dispatch gates the clause; persistent immutability makes prior
      history
;; a first-class value.
(defmethod record-reading :around
    [(sensor <sensor>) (reading (critical?))]
  (bind [history (get sensor :alert-history [])
         result  (call-next-method)]
    (assoc result :alert-history (conj history reading))))
```

Listing 5: Value-Based Alerting (FOL)

```
(defmethod notify [{:keys [(temp (>= 100))]}]
  (print "ALERT: Critical Temperature!"))
```

Predicate arguments are closed over at compile time: `(>= 100)` produces `(lambda (temp) (>= temp 100))`.

Comparison with Clojure and Common Lisp. The validated update guard requires three lines of FOL and is defined once. Clojure has no `:around` method equivalent; a wrapper function (Listing 6) requires updating every call site. Protocols or multimethods with a sentinel dispatch value can interpose without call-site changes, but require restructuring the dispatch hierarchy—the burden shifts from call sites to dispatch design. Common Lisp supports `:around` methods, but without persistent objects discarding an invalid update requires an explicit $O(N)$ copy before calling the primary (Listing 7).

Listing 6: Validated Update Guard in Clojure — callers must use wrapper

```
;; Guard is in a wrapper; every call site must be changed from
;; (update! obj key val) to (guarded-update! obj key val):
(defn guarded-update! [obj key val]
  (if (valid-update? obj key val)
    (update! obj key val)
    obj))
;; Call sites: (update! ...) -> (guarded-update! ...)  ; manual at
      each site
```

Listing 7: Validated Update Guard in Common Lisp — requires explicit copy

```
(defmethod update-obj :around ((obj guarded) key val)
  ;; No persistent objects: must copy before calling primary to
     preserve
  ;; original for potential discard.  Cost: O(N) in number of slots.
  (let ((saved (make-instance (class-of obj)
                 :slot-a (slot-value obj 'slot-a)
                 :slot-b (slot-value obj 'slot-b))))
    (if (valid-update-p obj key val)
        (call-next-method)
        saved)))
```

FOL centralises the guard with no call-site changes and discards the update in $O(1)$ because the pre-update snapshot is already an immutable first-class value.

4.1.5 Design Limitations of Predicate Dispatch. **Predicate side effects.** The $O(N)$ linear scan evaluates all non-matching predicates before finding the match; side effects (e.g., logging) fire once per non-matching clause. Level-3 predicates should be pure or explicitly idempotent.

Definition-order tie-breaking. Load-order sensitivity arises only when patterns are incomparable under $\prec$: same Level-3 category with no type relationship (e.g., `(prime?)` vs. `(fibonacci?)`), or structurally incomparable (map vs. sequential). Within a library, collapse overlapping Level-3 patterns into a single clause or demote one to Level 2. For cross-library conflicts, an explicit `:around` method resolves the ambiguity deterministically, independent of ASDF load order.

Limited static analysis. For `defmethod` (open dispatch), predicate specializers require runtime evaluation and cannot be cached; type checks use an $O(N)$ linear scan rather than a discriminating function, and unreachable clauses are detectable only at runtime. For `defn` (closed dispatch), the full clause set is known at compile time: detecting shadowed clauses—pairs (c_i, c_j) where $c_i \prec_{\text{disp}} c_j$ and c_i's guard subsumes c_j's—is in principle feasible without a theorem prover when both guards are Level-2 type checks (via Spec-Type) or when structural arity makes one strictly wider. We do not currently implement this analysis; it is near-term planned work.

4.2 Schema Evolution

Chain replay (`:after` on `update-instance-for-redefined-class`) walks the metaclass *version chain*, composing single-hop alias maps from birth to current, and recovers renamed values from discarded slot property-lists. Replay fires lazily at first post-redefinition access; all intermediate hops are composed in a single pass. This migration is confluent under the invariant that no slot name appears as both source and target across the *entire* version chain. The macro-expansion step enforces this globally: it checks cyclical renames within a single hop ($A \to B \to A$) *and* cross-hop cycles ($A \to B$ in hop 1, $B \to A$ in hop 2) by walking the accumulated alias map at each redefinition, signaling a compile-time error on any cycle or fan-in conflict (both X and Y renamed to Z). Under this invariant, alias maps compose as name-remapping functions with disjoint domain and range, so replaying hops in any order yields the same result. These invariants hold only for source-level `defclass` forms; programmatic class construction via `(eval '(defclass ...))` bypasses macro-expansion checks and is the programmer's responsibility to keep consistent.

Listing 8: Lazy Schema Migration with Aliasing

```
;; Schema 1
(defclass <sensor> [] [[id] [reading]])
;; Schema 2: rename reading -> val; :reading keyword still routes to
      val
(defclass <sensor> [] [[id] [val :alias reading]])
```

At first post-redefinition access, chain replay finds `:reading` in discarded slot property-lists and writes it into `:val`; afterwards `(get obj :reading)` resolves via the alias map, so call sites need no updates.

4.2.1 Persistence and Versioning Performance. Versioning overhead is $O(H \cdot A)$ (H = redefinitions, A = aliases per hop); in practice $H \cdot A \ll 50$. Replay cost is a few microseconds, paid once per dormant instance; subsequent reads proceed at the standard 1.4–2.1× rate.

5 Evaluation

5.1 Common Lisp and Clojure Comparison

FOL fills a gap between CLOS and Clojure (Table 5): where `defrecord` provides map-backed values, FOL provides MOP-integrated persistence with full CLOS method combinations.

5.2 Micro-Benchmarks

We measured persistent objects vs. mutable CLOS across slot counts. **Construction** costs ≈183 ns for a 2-slot object vs. ≈37 ns (mutable `make-instance`, 5×); `%make-persistent` bypasses `initialize-instance`, delivering a 4.8× speedup. **Reads** are 1.2–2.0× native; **Updates** are 30–246× slower (Table 6). $T = 8$ is optimal for the 8-slot case and achieves the minimum for $N \geq 32$; it yields to higher thresholds only at intermediate widths under-represented in practice [22].

Table 5: Built-in Feature Comparison

Feature	FOL	Clojure	CL
Static type safety	×	×	×
Persistent objects	✓[d]	○[c]	×
CLOS/MOP	✓	×	✓
Method combinations	✓	×	✓
Predicate dispatch	✓	single[a]	×[b]
Schema evolution	✓[e]	×	Manual
Transducers	✓	✓	×
Macros	✓	✓	✓
Raw scalar performance	Low[f]	High	High

[a] `defmulti` dispatches via a user-supplied function; `prefer-method`/`derive` allow manual overrides but there is no automatic multi-parameter specificity ordering.
[b] Multi-parameter type dispatch natively; libraries like `trivia` add pattern matching.
[c] Clojure has persistent *data structures*; `defrecord` is map-backed, not MOP-integrated.
[d] Slot storage backed by immutable HAMTs; `(setf slot-value)` blocked; updates via `assoc`.
[e] Renamed slots recovered automatically at first post-redefinition access; multi-hop chains composed in a single pass.
[f] Scalar arithmetic and non-slot operations inherit full CL performance. "Low" refers specifically to persistent object slot access: reads add 1.4–2.1× overhead and writes add 33–447× overhead relative to mutable CLOS.

Table 6: Actual slot update overhead (μs/op) vs. native CLOS

Slots	Persistent	Mutable	Ratio	Notes
2	0.58 μs	14.1 ns	41×	2 native slots
8	1.03 μs	13.4 ns	77×	8 native slots
32	1.08 μs	26.8 ns	40×	all-native (T=8 strategy)
48	2.23 μs	24.0 ns	93×[a]	32N + 16-slot trie
128	4.00 μs	16.1 ns	248×	32N + 96-slot trie

[a] Ratio drops vs. 32 slots because the mutable CLOS baseline is 46% slower at 48 slots (discontiguous memory layout), not because persistent overhead decreases.

Table 7: Modeled update cost (μs/op) at candidate thresholds T

N	T=8	T=12	T=16	T=24	T=32	T=8 strategy
8	**0.42**	0.42	0.43	0.42	0.42	all-native
12	0.75	**0.62**	0.62	0.61	0.60	8N + 4V
16	0.94	0.91	0.83	0.78	**0.77**	8N + 8V
24	**1.15**	1.23	1.28	**1.15**	**1.15**	8N + 16V
32	**1.39**	1.47	1.51	1.58	1.52	8N + 24V
48	**1.95**	1.99	2.04	2.12	2.22	8N + 40V

Bold = minimum in row. "kN + jV" = copy k native slots then batch-build j-element overflow vector. All-native = copy N slots directly. Simulated copy costs only; actual `update-slots` is 1.6–3× higher due to MOP iteration overhead (see Table 6).

5.2.1 Predicate Dispatch Micro-benchmark. We measured the per-invocation cost of a 20-method generic function, each clause specialized on a distinct Level-3 predicate, comparing FOL open and closed dispatch against standard CLOS type dispatch and a manual `cl:cond` cascade (Table 8, SBCL 2.6.0 x86-64).

Table 8: Predicate Dispatch Performance (N=20 clauses).

Dispatch Method	Time/op	Ratio (vs. CLOS)
Standard CLOS (Type)	37.4 ns	1.00×
Manual `cl:cond` Cascade	62.4 ns	1.66×
FOL `defn` (Closed Dispatch)	60.0 ns	1.60×
FOL `defgeneric` (Open Dispatch)	64.4 ns	1.72×

FOL's predicate dispatch adds ≈1.6× overhead, matching the manual `cl:cond` baseline; the 20-clause all-Level-3 setup is a worst case—functions with only Level-2 specializers incur no predicate-scan cost.

5.2.2 Lazy Evolution Micro-Benchmark. We measured single-hop and multi-hop migration against a CL baseline with a manual `update-instance-for-redefined-class` method. FOL first-touch cost is ≈2× CL's; multi-hop cost is flat (7.8–8.4 μs) as chain-replay composes alias maps in a single pass (Tables 9–10).

Table 9: Single-hop lazy migration: FOL vs. CL (μs/op, 10K instances)

Phase	FOL	CL	Notes
Migration (first touch)	6.40	3.30	one-time per dormant instance
of which: overhead vs. steady-state	5.63	3.28	SBCL migration + alias recovery
Steady-state update	0.77	0.02	post-migration, repeated use

Table 10: Multi-hop lazy migration first-touch cost (μs/op, 5K instances per cohort)

Hop depth	FOL	CL	Notes
1-hop (v2→v3)	8.19	4.10	FOL allocates new object; CL mutates in-place
2-hop (v1→v3)	3.57	5.62	extra plist scan (CL); extra hash-table pass (FOL)
3-hop (v0→v3)	4.48	5.13	both dominated by SBCL migration machinery
Steady-state	0.69	0.01	post-migration; uniform across all cohorts

5.3 Naive-Translation Workloads

Persistent *slot* updates are 33–447× slower than mutation (Table 6). Naive Quicksort and BFS (Listings 9–10) illustrate the path-copying penalty (Table 11).

Quicksort overhead stems from 3.4M path copies; transients cut this from 96× to 40× (−97% allocation). BFS improves similarly (39× → 6.2×).

Listing 9: Naive Persistent Partition

```
(defn partition [v low high]
  (bind [pivot (get v high)]
    (loop [j low i (dec low) curr-v v]
      (if (< j high)
          (if (<= (get curr-v j) pivot)
              (bind [next-i (inc i)
                     v1 (assoc curr-v next-i (get curr-v j))
                     v2 (assoc v1 j (get curr-v next-i))]
                (recur (inc j) next-i v2))
              (recur (inc j) i curr-v))
          (bind [next-i (inc i)
                 v1 (assoc curr-v next-i (get curr-v high))
                 v2 (assoc v1 high (get curr-v next-i))]
            [v2 next-i])))))
```

Listing 10: Naive Persistent BFS Step

```
(defn bfs-step [q dists graph]
  (if (empty? q)
      dists
      (bind [u (first q)
             edges (get graph u)
             d (get dists u)]
        (loop [es edges nq (rest q) nd dists]
          (if (empty? es)
              (bfs-step nq nd graph)
              (bind [v (first es)]
                (if (= (get nd v) -1)
                    (recur (rest es)
                           (conj nq v)
                           (assoc nd v (inc d)))
                    (recur (rest es) nq nd))))))))
```

Table 11: Naive-Translation Workload Results

Workload	Implementation	Time	Alloc	vs. CL
QuickSort	CL (simple-vector)	12 ms	0.8 MB	1.0×
$N = 100{,}000$	FOL Persistent (assoc)	1199 ms	2598 MB	∼96×
	FOL Transient (assoc!)	494 ms	91 MB	∼40×
BFS	CL (hash-table)	4 ms	2.3 MB	1.0×
$N = 50{,}000$	FOL Persistent (assoc)	148 ms	119 MB	39×
	FOL Transient (assoc!)	24 ms	13 MB	6.2×

Idiomatic Reformulations. Algorithmic restructuring eliminates the translation penalty (Table 12). **Idiomatic merge sort** recurses on non-overlapping halves via `subvec` with successive `conj` calls. **BFS with lazy initialization** adds each node on first discovery, eliminating N up-front `assoc` calls.

Merge sort pays $1.7M\ O(\log N)$ trie-node touches (near the $O(N \log^2 N)$ theoretical minimum); transients are counterproductive because the per-`conj` path is already shallow. Lazy BFS eliminates initialization waste; transients help further (29× → 14×).

5.4 Abstract Syntax Tree Rewriting Engine

To evaluate persistent objects and predicate dispatch together, we implemented an AST optimization engine. The benchmark defines 25 persistent node classes: one abstract base, two leaf types (`<op-var>`, `<op-lit>`), and 23 binary operator classes. A single

Table 12: Idiomatic Algorithm Results: persistent and transient FOL vs. native CL

Workload	Implementation	Time	Alloc	vs. CL
Merge Sort	CL (simple-vector)	15 ms	1.6 MB	1.0×
$N = 100{,}000$	FOL persistent (`conj`)	457 ms	623 MB	30×
	FOL transient (HAMT array)	491 ms	81 MB	32×[a]
BFS lazy	CL (hash-table)	4 ms	2.3 MB	1.0×
$N = 50{,}000$	FOL persistent (lazy dict)	103 ms	61 MB	29×
	FOL transient (`hamt-assoc!`)	51 ms	16 MB	14×

[a] Transient is slower than persistent here: persistent `conj` touches only a root-to-leaf path of depth ≤ 5 per append, so transient setup cost is not recovered at $N = 100{,}000$.

multi-clause `defmethod` encodes 54 algebraic rewrite rules via predicate dispatch, including nested structural patterns such as the identity-element rule:

```
(defmethod ast-optimize
  [({:keys [(left ({:keys [(val (eql 0))]} <op-lit>))
            (right r)]} <op-add>)]
  r)
```

A second 25-clause `walk` method drives recursive descent. The test tree is a 9,999-node balanced binary tree (depth 13) with operators cycling uniformly through all 23 classes. The CL baseline uses `defstruct` with a `typecase` cascade—the most favourable reasonable CL implementation.

Table 13: AST Optimizer: FOL vs. CL ($N = 100$ passes, 9,999-node balanced tree).

Phase	Implementation	Time	Alloc	vs. CL
Build	CL	0.398 ms	0.16 MB	1.0×
	FOL	31.1 ms	12.7 MB	78×
Optimizer	CL	0.240 ms/pass	0.125 MB/pass	1.0×
(per pass)	FOL	10.36 ms/pass	1.749 MB/pass	43×

FOL is 47× slower per optimizer pass (14× more memory), from three sources: **dispatch routing** (≈6×, worst case: uniform distribution across all 25 clauses); **field reads** (≈50 ns vs. ≈4 ns for struct, 12×); **construction** (≈183 ns vs. ≈37 ns, 5×). The build ratio (78×) exceeds the optimizer ratio because build requires one `make-<class>` per node with no amortization; speculative rewrite passes are $O(1)$ to roll back.

5.5 Logic Simulation (LSim)

LSim is an event-driven digital logic simulator that exercises persistent data structures at scale. To isolate the event-queue cost, we hold gate-connectivity constant: the **CL** baseline uses a mutable destructive leftist heap [8] ($O(1)$ allocation per merge); **FOL** uses a persistent leftist heap [25] ($O(\log n)$ copies). Table 14 covers six configurations spanning five orders of magnitude; the largest circuits used a 56 GB heap (`-dynamic-space-size 57344`).

Normalised by gate evaluations, FOL converges to ≈11.4 μs/eval while CL degrades to ≈44.0 μs/eval; the persistent heap's $O(\log n)$ allocation yields better cache locality than the fragmented mutable heap at extreme scale.

Table 14: LSim PQ Scaling Results (mean CPU time).

Config	Gate Evals	CL	FOL	Ratio
8bit-100	876	78 ms	109 ms	1.4×
32bit-300	10 224	94 ms	141 ms	1.5×
8x32-9000	281 248	359 ms	1.87 s	5.2×
32x32-3000	3 850 304	7.57 s	34.15 s	4.5×
8x32x32-9000	89 708 032	906 s	983 s	1.1×
32x32x32-30000	1,183,510,528	52,050 s	13,531 s	0.26×

A 2×2 ablation (Table 15) attributes overhead between the event-queue and simulation-state.

Table 15: 2×2 Ablation: Mutable vs. Persistent Heap × State Maps (mean wall time; ratios relative to CL baseline).

Circuit	CL	Hybrid-A (persist. maps)	Hybrid-B (persist. heap)	FOL
32bit-300	94 ms	141 ms (1.5×)	125 ms (1.3×)	141 ms (1.5×)
8x32-9000	359 ms	1.87 s (5.2×)	2.25 s (6.3×)	1.87 s (5.2×)
32x32-3000	7.57 s	34.1 s (4.5×)	42.7 s (5.6×)	34.1 s (4.5×)
8x32x32-9000	906 s	983 s (1.1×)	1646 s (1.8×)	983 s (1.1×)

Persistent maps add 2.0–2.6×; **persistent heap** overhead falls from 3.6× to 1.9× at scale; their interaction is sub-multiplicative (allocation pressure overlaps in the same nursery generation). At the largest configuration (32x32x32-30000), FOL runs 3.8× faster than the CL baseline (0.26× ratio). This result is driven primarily by *CL degrading*: the mutable heap fragments under 1.18 billion merge operations, while FOL's $O(\log n)$ allocation maintains stable throughput (≈11.4 μs/eval); CL deteriorates to ≈44.0 μs/eval. A CL implementation using a slab allocator or an explicit free-list for cons cells could reduce fragmentation and narrow the gap; we do not currently have such a baseline, and establishing one is future work.

5.6 CLOS Adoption Cost: Expression Interpreter

The AST-optimizer and LSim benchmarks were designed to exercise FOL's key value propositions. A complementary question is: *what does it cost to port an ordinary CLOS codebase to FOL?* We implement an expression interpreter over seven node types using idiomatic CLOS as baseline, then port it to FOL with minimal structural changes.

Setup. The interpreter evaluates arithmetic expressions over seven persistent node classes: `<num-expr>`, `<var-expr>`, `<add-expr>`, `<mul-expr>`, `<let-expr>`, `<if-expr>`, and `<neg-expr>`. Three generic functions are defined without modifying the class definitions—demonstrating the "new operations" dimension of the Expression Problem [32]: `eval-expr`, `pretty`, and `free-vars`.

The key structural difference between the two implementations is the environment representation in `eval-expr`. The CL version uses an idiomatic *shadow-and-restore* pattern: the hash-table is mutated in place to bind a variable, the body is evaluated, and the old value is restored. The FOL version uses `(assoc env bvar v)`, returning a new persistent dict with structural sharing—no mutation, no bookkeeping. All other structural differences are mechanical (slot readers, string building, set literals), and the method bodies are otherwise identical.

A corpus of 50 depth-5 expression trees (all seven node types in realistic proportions) is evaluated across 1,000 iterations (150,000 total generic-function call sequences) via `eval-expr`, `pretty`, and `free-vars`.

Table 16: Expression Interpreter Benchmark (SBCL 2.6.0, AMD Ryzen 9 5900X).

Phase	CL	FOL	Time ratio	Mem
Build (50 trees, depth 5)	9.6 ms / 3.8 MB	19.9 ms / 8.9 MB	2.1×	
Eval (1000 iter × 50 exprs)	0.27 s / 224.5 MB	1.88 s / 1300 MB	7.0×	

Results. The 7× eval ratio (vs. 47× for the AST optimizer) reflects: a `make-instance` baseline (≈2× construction vs. 5× for `defstruct`), 7-clause scans rather than 25, and slot-read-heavy dispatch.

What the port required. The structural shape of every method is identical between the two versions: one clause per node type, same recursion pattern, no architectural changes. The mechanical differences are: (i) `defclass` syntax and automatic persistent metaclass; (ii) slot reads; (iii) constructors; (iv) let-binding environment (6-line shadow-and-restore → 3-line `bind`/`assoc`); (v) string and set APIs (`concatenate`/`member`/`union` → `str`/`contains?`/`union` on persistent sets).

This benchmark directly answers the adoption-cost question: porting idiomatic CLOS to FOL incurs a ≈7× runtime overhead at this scale, with no required architectural changes to the existing class hierarchy.

6 Threats to Validity

Single implementation: All measurements use SBCL 2.6.0; results may differ on other CL implementations. **Experimenter bias**: FOL and all CL baselines were written by the same author; a more aggressive CL implementation could narrow reported ratios. **Benchmark coverage**: Small LSim configs ×20, medium ×3; the 32×32×32-30000 circuit was run once per implementation (CL: ≈52,000 s, FOL: ≈13,500 s). **Benchmark selection bias**: The first two macro-benchmarks were designed with FOL's key trade-offs in mind and may over-represent scenarios where FOL's value proposition is realized; the interpreter and naive-translation benchmarks provide a more conservative picture. **Expressiveness comparisons are qualitative**: the code-level comparisons in Section 4 count lines and call-site changes but do not measure programmer productivity; a controlled user study remains future work. **CL interoperability hazard**: CL library authors who extend FOL classes with standard `defmethod` clauses may be surprised to find that `(setf slot-value)` on a persistent object raises a runtime error and `change-class` is blocked. These restrictions follow from the persistence invariant (Section 2) but are not signalled at method-definition time. **Potential non-portability**: The `persistent-class` metaclass relies on the MOP hooks listed in Table 1; portability to non-SBCL runtimes (e.g., CCL, ECL, ABCL) is untested.

7 Related Work

Persistent data structures. FOL's trie implementation follows Bagwell's HAMT [3] and Clojure's vector and map designs [16], building on path-copying persistence theory [10]; Okasaki's purely functional heaps [25] underpin the persistent leftist heap used in LSim. The closest Common Lisp precedents are Sycamore [9] and FSet [5], functional set libraries; FOL extends this foundation to MOP-integrated persistent *objects*, adding predicate dispatch and schema evolution—capabilities neither provides.

Schema evolution. Gemstone [28] pioneered lazy migration for persistent Smalltalk objects, requiring an explicit conversion method per rename. Atkinson and Morrison [2] survey eager/lazy migration strategies; in all cases studied, explicit per-rename coercion code is required. ENCORE [30] introduced per-redefinition class versioning with first-access migration, anticipating FOL's lazy strategy. FOL's contribution: migration is *declarative* via `:alias` rather than procedural coercion; multi-hop chains compose in a single pass; and the mechanism requires no runtime changes beyond the standard CLOS MOP.

OOP/FP bridges and pattern dispatch in other languages. Scala's case classes [24] and `match` [11] provide immutable ADTs with structural dispatch, but updates require explicit `copy` and the class system is not MOP-integrated. Racket [13] separates `match` from its class system [14]; Julia [4] provides multi-parameter type dispatch (FOL's Level 2) but no predicate specializers and mutable objects.

Expression Problem solutions. Object algebras [26] and tagless-final style [6] solve both dimensions in typed functional languages without subtyping or casts; both require more ceremony than FOL's multimethods for the "new operations" dimension but provide static extensibility guarantees FOL lacks. FOL's approach—open generic functions over persistent classes—corresponds to the multimethod solution studied by Zenger and Odersky [33] and trades static safety for CLOS expressibility.

Predicate and multiple dispatch. Predicate dispatch was pioneered in Cecil [7] and formalized as a unified theory by Ernst et al. [12]; both require theorem-prover subsumption for method ordering. FOL deliberately sacrifices semantic exactness for syntactic tractability: its level ordering is a conservative approximation that avoids the $O(N^2)$ subsumption cost at the price of the type-annotated conjunction deficiency noted in Section 4. JPred [23] shows that typed predicate dispatch can recover ordering for type-annotated guards without a full theorem prover, by requiring each predicate method to declare its type domain and checking containment at load time. FOL does not currently adopt this approach for two reasons: (i) CLOS's open-world model allows new subclasses to be defined after any method has loaded; load-time containment checks would need re-evaluation on every subsequent `defclass`, making the cost unbounded in a live image; (ii) JPred's modularity guarantees assume a closed-world class hierarchy, which conflicts with CLOS's open-world `defmethod` extension model. The proposed future composite level (Section 4) — type-annotated predicates of the form `(x <integer> (prime?))` — is a restricted form inspired by JPred that would be compatible with CLOS's open world by treating the type annotation as a runtime guard rather than a compile-time constraint. GHC's overlapping instances [15] order type-class instance heads via `OVERLAPPING`/`OVERLAPPABLE` pragmas but are restricted to type-level patterns with no predicate stratum or schema evolution. Filtered dispatch [27] introduced arbitrary guard predicates as first-class dispatch criteria, directly motivating FOL's Level-3 stratum. Dylan's singleton specializers [1] and Slate's prototype-based multiple dispatch [29] each correspond to restricted forms of FOL's approach. Clojure's `defmulti` [16] dispatches on the return value of a user-supplied function and resolves ambiguity via `prefer-method`. Clojure's *protocols* [18] dispatch on the first argument's class only, with no predicate specialization; they are faster than `defmulti` for the common single-type case. FOL's `defmethod` occupies the same niche as `defmulti`—open, multi-parameter, extensible—but adds predicate specializers and CLOS method combinations, at the cost of an $O(N)$ linear scan rather than `defmulti`'s dispatch-function call. Self [31]'s "slots as map entries" model influences FOL's object representation; FOL retains a class hierarchy and MOP dispatch rather than prototype delegation, and adopts Clojure's epochal time model [17] for coordinated state change.

8 Conclusion

We presented two contributions to the design of Lisp object systems. First, a **persistent metaclass system** built on the CLOS MOP: hybrid native/trie storage keeps ≈80% of classes on the 1.4–2.1× read-overhead native path; lazy alias-based schema migration delivers transparent instance evolution; and `%make-persistent` reduces object creation to 5× over mutable `make-instance`. Second, a **specificity-ordered predicate dispatch** system whose load-time-decidable $\prec_{\text{disp}}$ orders all structurally or type-differentiated patterns automatically, confining load-order sensitivity to incomparable Level-3 predicates.

Persistent slot updates cost 30–248× in isolation; at 1.18 billion gate evaluations FOL runs 3.8× faster as the CL heap degrades, while the persistent heap maintains stable $O(\log n)$ throughput. The AST optimizer decomposes the overhead: ≈6× dispatch routing, ≈12× field reads, ≈5× construction; $O(1)$ speculative rollback is available at no extra cost.

Open problems include: native compilation to replace the $O(N)$ predicate scan; static clause analysis to detect unreachable patterns at load time; and coordinated value transitions across agent networks with snapshot consistency.

FOL's implementation is available at
https://github.com/frankadrian314159/fol.

References

[1] Apple Computer, Inc. 1992. *Dylan - An Object-Oriented Dynamic Programming Language.* Apple Computer, Inc.

[2] Malcolm Atkinson and Ron Morrison. 1995. Orthogonally Persistent Object Systems. *The VLDB Journal* 4, 3 (1995), 319–401.

[3] Phil Bagwell. 2001. *Ideal Hash Trees.* Technical Report 1. EPFL, Lausanne, Switzerland.

[4] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. 2017. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev.* 59, 1 (2017), 65–98.

[5] Scott L Burke. 2007. FSet: A functional set-theoretic collections library. https://common-lisp.net/project/fset/

[6] Jacques Carette, Oleg Kiselyov, and Chung-chieh Shan. 2009. Finally Tagless, Partially Evaluated: Tagless Staged Interpreters for Simpler Typed Languages. In *Journal of Functional Programming*, Vol. 19. 509–543.

[7] Craig Chambers. 1993. The Cecil Language: Design and Performance. In *Proceedings of the Conference on Object-Oriented Programming Systems, Languages, and Applications.* 243–257.

[8] Clark Allen Crane. 1972. *Linear Lists and Priority Queues as Balanced Binary Trees.* Technical Report STAN-CS-72-259. Stanford University.

[9] Neil T. Dantam. 2013. Sycamore: Purely Functional Data Structures for Common Lisp. https://github.com/ndantam/sycamore

[10] James R Driscoll, Neil Sarnak, Daniel D Sleator, and Robert E Tarjan. 1989. Making Data Structures Persistent. *J. Comput. System Sci.* 38, 1 (1989), 86–124.

[11] Burak Emir, Martin Odersky, and John Williams. 2007. Matching Objects with Patterns. In *ECOOP 2007–Object-Oriented Programming*. Springer, 273–298.

[12] Michael D Ernst, Craig Kaplan, and Craig Chambers. 1998. Predicate Dispatching: A Unified Theory of Dispatch. In *ECOOP'98—Object-Oriented Programming*. Springer, 186–211.

[13] Matthew Flatt et al. 2015. The Racket Manifesto. In *20 Years of the Past and 20 Years of the Future (Snapshots), Dagstuhl Seminar 15451*. https://doi.org/10.4230/DagRep.5.11.112

[14] Matthew Flatt, Robert Bruce Findler, and Matthias Felleisen. 2006. Scheme with Classes, Mixins, and Traits. In *Asian Symposium on Programming Languages and Systems*. Springer, 270–289.

[15] GHC Team. 2004. *GHC User's Guide: Overlapping Instances*. Glasgow Haskell Compiler. https://downloads.haskell.org/ghc/latest/docs/users_guide/.

[16] Rich Hickey. 2008. The Clojure programming language. In *Proceedings of the 2008 symposium on Dynamic languages*. 1.

[17] Rich Hickey. 2009. Are We There Yet? JVM Languages Summit Keynote. https://github.com/matthiasn/talk-transcripts/blob/master/Hickey_Rich/AreWeThereYet.md

[18] Rich Hickey. 2010. Clojure Protocols. Clojure Reference Documentation. https://clojure.org/reference/protocols

[19] Rich Hickey. 2010. Clojure's Solutions to the Expression Problem. Conference talk at Strange Loop. Available at [thestrangeloop.com](https://www.thestrangeloop.com/2010/clojures-solutions-to-the-expression-problem.html.

[20] Rich Hickey. 2014. Transducers. Clojure Blog. https://clojure.org/reference/transducers

[21] Gregor Kiczales, Jim Des Rivieres, and Daniel Gureasko Bobrow. 1991. *The Art of the Metaobject Protocol*. MIT press.

[22] Michele Lanza and Radu Marinescu. 2006. *Object-Oriented Metrics in Practice*. Springer.

[23] Todd Millstein. 2004. Practical Predicate Dispatch. In *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*. ACM, 345–364.

[24] Martin Odersky, Lex Spoon, and Bill Venners. 2016. *Programming in Scala, 3rd Edition*. Artima Press.

[25] Chris Okasaki. 1998. *Purely Functional Data Structures*. Cambridge University Press.

[26] Bruno C. d. S. Oliveira and William R. Cook. 2012. Extensibility for the Masses: Practical Extensibility with Object Algebras. In *Proceedings of the 26th European Conference on Object-Oriented Programming (ECOOP)*. Springer, 2–27.

[27] Doug Orleans. 2002. Filtered Dispatch. In *Proceedings of the 17th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 20–26.

[28] D. J. Penney and J. Stein. 1987. Class Modification in the GemStone Object-Oriented DBMS. In *Proceedings of the 2nd ACM Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*. ACM, 111–117.

[29] Lee Salzman and Jonathan Aldrich. 2005. Prototypes with Multiple Dispatch: An Expressive and Dynamic Object Model. In *ECOOP 2005–Object-Oriented Programming*. Springer, 312–336.

[30] Andrea H. Skarra and Stanley B. Zdonik. 1986. The Management of Changing Types in an Object-Oriented Database. In *Proceedings of the 1st ACM Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*. ACM, 483–495.

[31] David Ungar and Randall B. Smith. 1987. Self: The Power of Simplicity. *ACM SIGPLAN Notices* 22, 12 (1987), 227–242.

[32] Philip Wadler. 1990. Linear Types Can Change the World!. In *Programming Concepts and Methods*.

[33] Matthias Zenger and Martin Odersky. 2005. Independently Extensible Solutions to the Expression Problem. In *Proceedings of the 12th International Workshop on Foundations of Object-Oriented Languages (FOOL)*.